Reviewer Pack — Tree-Depth Exponentiates Resolution Length for Tseitin Formu…

Entry e45f237e451d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 06:10:32 UTC

Tree-Depth Exponentiates Resolution Length for Tseitin Formulas

Entry ID: `e45f237e451d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 06:10:32 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics (Tree-depth parameter) **Field B** (complexity object): Resolution Length of Tseitin Formulas

Statement:

> For any graph G, the resolution length of the Tseitin formula on G is at least 2^{tree-depth(G)}

Rationale (proposer's reasoning):

> Tree-depth measures the complexity of a graph's structure, and higher tree-depth may correlate with longer resolution proofs due to the need for more steps to resolve clauses

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7ad13acc35a72d66`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def tree_depth(graph):
    n = len(graph)
    visited = [False] * n
    depth = 0

    def dfs(node, current_depth):
        nonlocal depth
        if current_depth > depth:
            depth = current_depth
        visited[node] = True
        for neighbor in graph[node]:
            if not visited[neighbor]:
                dfs(neighbor, current_depth + 1)

    for i in range(n):
        if not visited[i]:
            dfs(i, 1)

    return depth

def resolution_length(clauses):
    literals = set()
    for clause in clauses:
        literals.update(clause)

    unit_clauses = {l: [] for l in literals}
    for clause in clauses:
        if len(clause) == 1:
            unit_clauses[clause[0]].append(clause)

    resolution_steps = 0
    while True:
        new_unit_clauses = {}
        found_new_clause = False

        for literal, unit_clauses_list in unit_clauses.items():
            for clause in unit_clauses_list:
                if -literal in literals:
                    new_literal = -literal
                    new_clause = [l for l in clause if l != literal]
                    new_clause.append(new_literal)
```

```
                    if new_clause not in clauses:
                        clauses.add(tuple(sorted(new_clause)))
                        found_new_clause = True
                        new_unit_clauses[new_literal] = new_unit_clauses.get(new_literal, []) + [r

        if not found_new_clause:
            break

        unit_clauses = new_unit_clauses
        resolution_steps += 1

    return resolution_steps

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    graph = [[] for _ in range(n)]
    for _ in range(random.randint(int(0.5 * n * (n - 1)), int(0.8 * n * (n - 1)))):
        u, v = random.sample(range(n), 2)
        if v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)

    clauses = []
    for i in range(n):
        for j in range(i + 1, n):
            clauses.append((i, -j))
            clauses.append((-i, j))

    depth = tree_depth(graph)
    length = resolution_length(set(tuple(sorted(c)) for c in clauses))

    return {
        "metric_name": "resolution_length",
        "metric_value": length,
        "instances_tested": 1,
        "conjecture_holds": length >= 2 ** depth,
        "counterexample": "" if length >= 2 ** depth else f"Graph with n={n}, A={graph}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1000, 9999) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    std_length = math.sqrt(sum((r["metric_value"] - mean_length) ** 2 for r in results) / len(resu
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
```

```
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fr
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE no_counterexamples")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
 16, 12, 20], [16, 3, 13, 15, 10, 6, 0, 5, 9, 11, 20, 19], [9, 0, 11, 1, 12, 5, 2, 17, 14, 19, 10, 6, 18, 1
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_ho
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 0, 'instances_tested': 1, 'conjecture_ho
TRIAL: {'metric_name': 'resolution_length', 'metric_value': 0, 'instances_tested':
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed before producing conclusive data; pre-registered support condition cannot be evaluated. | next: Re-run test with increased time/memory limits and debug crash logs

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	125950
2	propose	ollama_remote	qwen3:8b	0	0	121943
3	preregistration	ollama_remote	qwen3:8b	0	0	31644
4	novelty	ollama_remote	qwen3:8b	0	0	27236
5	novelty	ollama_remote	qwen3:8b	0	0	22077
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15122
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9297
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8257
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10409
10	judge	ollama_remote	qwen3:8b	0	0	23851

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 395787 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e45f237e451d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e45f237e451d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e45f237e451d.tar.gz` (if generated)